Mechanized *and Automated* Verification of *Probabilistic Programs*

Christoph Matheja

DTU

Carl von Ossietzky Universität Oldenburg

DANMARKS FRIE FORSKNINGSFOND
INDEPENDENT RESEARCH FUND DENMARK

joint work with Kevin Batz, Oliver Bøving, Benjamin Kaminski, Joost-Pieter Katoen, Philipp Schröer

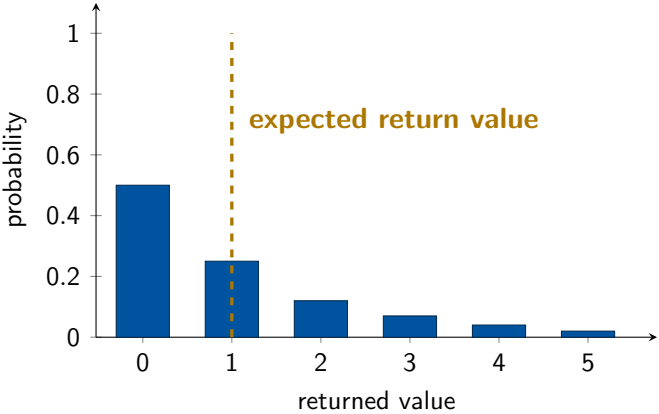
Meeting Oldenburg – Groningen

March 9, 2026

What are probabilistic programs (PPs)?

Probabilistic program = ordinary program + sampling from probability distributions

```
fn geo() -> int {
  coin := flip();
  if (coin = heads) {
    return 0
  } else {
    return 1 + geo()
  }
}
```



What are probabilistic programs good for?

Universal modeling formalism

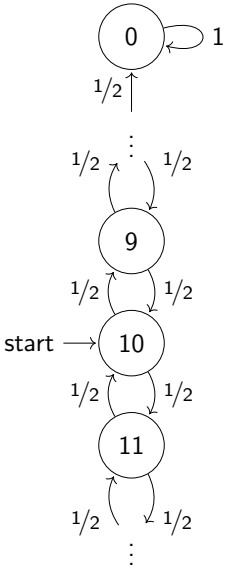
- Randomized algorithms
- Various kinds of (infinite-state) Markov models
- Communication and security protocols
- Bayesian networks, statistical models, ...

Typical analysis problems

- Bounding probabilities of temporal properties
- Expected resource usage
- Sensitivity analysis, higher moments, ...,

Example: Random Walk

```
x := 10;
while (x ≠ 0) {
  if (flip()) {
    x := x − 1
  } else {
    x := x + 1
  }
}
```



Termination probability: 1

Expected runtime: ∞

Example: Probabilistic Termination Phenomena

```
fn foo() -> int {
  if (flip() = heads) {
    return 0
  } else {
    return 1 + foo()
             + foo()
             + foo()
  }
}
```

What is the probability that *foo* terminates?

1 (almost-sure)

1

$\frac{\sqrt{5}-1}{2}$

Proving almost-sure termination on *one* input is as hard as proving that an ordinary program terminates on *all* inputs [Acta Inf. 2019]

Proof rules for reasoning about PPs (highly incomplete)

- **Expectation transformers**

  [Kozen 1983] [McIver & Morgan 2005] [POPL 2019-2023] [CAV 2021]
  [Batz et al. 2025] [Enea et al. 2026] ...

- **Supermartingales**

  [Chakarov et al. 2013] [Chatterjee et al. 2017-2025] [McIver et al. 2017]
  [Takisaka et al. 2021] [Abate et al. 2024, 2025]

- **Probabilistic Hoare logics**

  [den Hartog 2002] [Barthe et al. 2016-2025] [Bao et al. 2025]
  [Aguirre et al. 2023-2026]

AuRoRa Project Goal

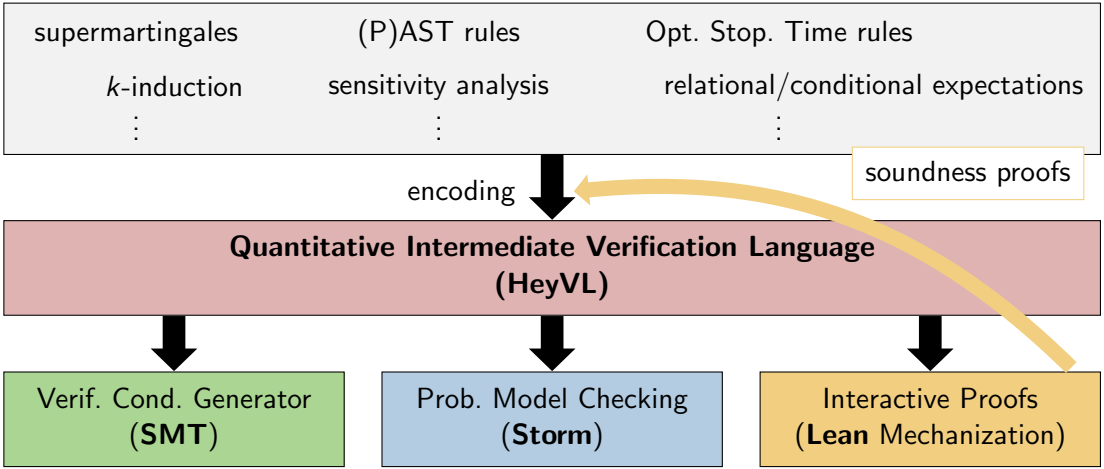
Develop an **intermediate language** for probabilistic program verification techniques

- $\rightsquigarrow$ Support feature-rich probabilistic programs
- $\rightsquigarrow$ Building efficient *automated* verifiers
- $\rightsquigarrow$ *Mechanization* for soundness proofs and interactive reasoning

Who is such a language for?

- Developers of proof rules $\rightsquigarrow$ rapid prototyping $\rightsquigarrow$ [Enea et al., POPL 2026]
- Developers of verification tools
  - $\rightsquigarrow$ Provide new verification backends
  - $\rightsquigarrow$ Utilize mechanization for proving new verification techniques sound
- Practitioners $\rightsquigarrow$ combine and adapt verification techniques

Plan: A Verification Infrastructure for Probabilistic Programs

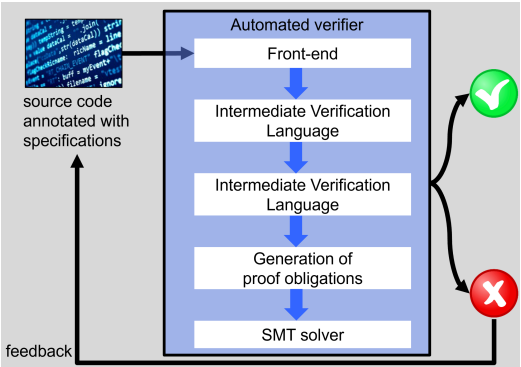


Inspiration: Classical Intermediate Languages à la Boogie

Idea: Build verifiers like compilers using a language for verification problems

- **Assertions** φ, ψ**:** first-order logic
- **Commands** C in intermediate language
- **Verification condition:** $\mathrm{wp}[C](\texttt{true})$ valid

C	$\mathrm{wp}[C](\varphi)$
$\texttt{assert}\ \psi$	$\psi \wedge \varphi$
$\texttt{assume}\ \psi$	$\psi \Rightarrow \varphi$
$\texttt{havoc}\ x$	$\forall x\colon \varphi$
$C_1\,;\ C_2$	$\mathrm{wp}[C_1](\mathrm{wp}[C_2](\varphi))$
$C_1\ \texttt{[]}\ C_2$	$\mathrm{wp}[C_1](\varphi) \wedge \mathrm{wp}[C_2](\varphi)$



Starting point: Weakest Preexpectations

[Kozen, 1983] [McIver & Morgan, 2005]

Why?

- All previous examples have been verified with expectation-based calculi
- Covers many supermartingales [McIver et al., 2017] [Takisaka et al., 2021]

Expectations

Program states: $\text{States} \;=\; \{\sigma \mid \sigma\colon \text{Vars} \to \mathbb{Q}\}$

Expectations: $\mathbb{E} \;=\; \{f \mid f\colon \text{States} \to \mathbb{R}^{\infty}_{\geq 0}\}$ think: **random variable**

$$f \;\preceq\; g \qquad \text{iff} \qquad \forall \sigma \in \text{States}\colon \quad f(\sigma) \;\leq\; g(\sigma)$$

Examples:

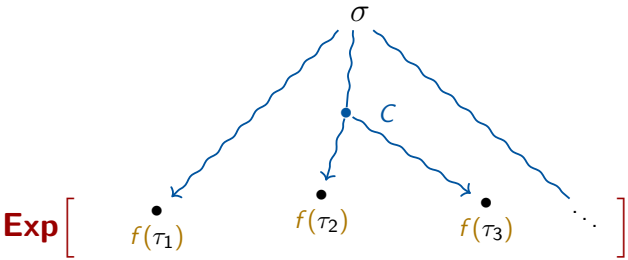
$$1 \quad = \quad \lambda\sigma.\ 1$$

$$[x < 10] \quad = \quad \lambda\sigma. \begin{cases} 1, & \text{if } \sigma \models x < 10 \\ 0, & \text{otherwise} \end{cases}$$

$$x^2 \quad = \quad \lambda\sigma.\ \sigma(x)^2$$

The Weakest Preexpectation

Given: probabilistic program C and postexpectation $f : \text{States} \to \mathbb{R}^{\infty}_{\geq 0}$
Running C on initial state σ yields a (sub-)distribution $[\![C]\!](s)$ over final states
Question: What is the **expected value** of f after termination of C?



$$\text{wp}[C](f) \;=\; \lambda\sigma. \int_{[\![C]\!](\sigma)} f \;\in\; \mathbb{E} \quad = \{f \mid f : \text{States} \to \mathbb{R}^{\infty}_{\geq 0}\}$$

Examples

postexpectation f	**weakest preexpectation** $\mathrm{wp}[C](f)$
1	probability that C terminates
$[x < 10]$	probability that $x < 10$ holds upon termination
x^2	expected value of x^2 after termination of C

The weakest preexpectation calculus for pGCL

$\mathrm{wp}[C](f)$: expected value of f after termination of C evaluated in initial states

C	$\mathrm{wp}[C](f)$
skip	f
$x := \mu$	$\lambda\sigma. \sum_{v \in \mathbb{Q}} \mu(\sigma)(v) \cdot f[x \mapsto v](\sigma)$
$C_1 \,;\, C_2$	$\mathrm{wp}[C_1](\,\mathrm{wp}[C_2](f)\,)$
if (b) { C_1 } **else** { C_2 }	$[b] \cdot \mathrm{wp}[C_1](f) \;+\; [\neg b] \cdot \mathrm{wp}[C_2](f)$
$\{C_1\}$ [p] $\{C_2\}$	$p \cdot \mathrm{wp}[C_1](f) \;+\; (1-p) \cdot \mathrm{wp}[C_2](f)$
while (b) { C }	$\mathit{lfp}(\Phi_f)$, where $\underbrace{\Phi_f(X) \stackrel{\text{def}}{=} [b] \cdot \mathrm{wp}[C](X) \;+\; [\neg b] \cdot f}_{\text{characteristic function of the loop}}$

Towards a verification infrastructure for probabilistic programs

1. **What are quantitative assertions?**
2. **What is an intermediate language for probabilistic program verification?**
3. **What can be encoded in such a language?**
4. **Status of Mechanization and Automation**

Syntactic Expectations

Classical verification:

$$\mathit{Pre} \models \mathrm{wp}[C](\mathit{Post})$$

Probabilistic verification:

$$g \preceq/\succeq \mathrm{wp}[C](f)$$

Theorem (Cook, 1978)

If $C \in$ GCL and $\mathit{Post} \in$ FO-arithmetic then

$$\mathrm{wp}[C](\mathit{Post}) \in \text{FO-arithmetic.}$$

Expressiveness for expectations?

If $C \in$ pGCL and $f \in$ Exp then

$$\mathrm{wp}[C](f) \in \text{Exp.}$$

What is an expressive syntax Exp for expectations $\mathbb{E} = \{f \mid f\colon \text{States} \to \mathbb{R}^{\infty}_{\geq 0}\}$?

Towards a sensible syntax

Requirement: $[b] \in \mathrm{Exp}$ for every Boolean expression b

$$
\begin{array}{l}
/\!/\!/ \ \dfrac{\sqrt{5}-1}{2} \notin \mathbb{Q}_{\geq 0} \\
x := 1; \\
\texttt{while}\,(\,x > 0\,)\,\{ \\
\quad \{x := x + 2\} \ \ [1/2] \ \ \{x := x - 1\} \\
\} \\
/\!/\!/ \ [\texttt{true}] \ = \ 1 \in \mathbb{Q}_{\geq 0}
\end{array}
$$

$\rightsquigarrow$ A sensible syntax must cover irrational and non-algebraic numbers

An Expressive Syntax for Expectations

$$
\begin{array}{rcll}
\varphi & ::= & a & \text{(arithmetic expressions over rational variables)} \\
 & | & [b] \cdot \varphi & \text{(guarding with Boolean expressions)} \\
 & | & a \cdot \varphi & \text{(rescaling with arithmetic expressions)} \\
 & | & \varphi + \varphi & \text{(addition of expectations)} \\
 & | & \mathcal{S} x.\ \varphi & \text{(supremum quantifier over variable } x\text{)} \\
 & | & \mathcal{I} x.\ \varphi & \text{(infimum quantifier over variable } x\text{)}
\end{array}
$$

Example:

$$\mathcal{S} x.\ 3 \cdot [x \cdot x < 2] \cdot x \quad = \quad 3 \cdot \sqrt{2}$$

Examples of expressible expectations

Polynomials	$x^2 + 3 \cdot y + 4$	(appear in martingale-based reasoning)
Rational functions	$\dfrac{x^2 + 3 \cdot y + 4}{2 \cdot x + y}$	(appear in analysis of probabilistic models)
Harmonic numbers	$\sum_{k=1}^{x} {}^{1}\!/_{k}$	(appear in runtime analysis of randomized algorithms)

Relative Completeness

Theorem (Expressiveness, POPL 2021)

If $C \in \text{pGCL}$ and $\varphi \in \text{Exp}$, one can construct a syntactic expectation $\psi \in \text{Exp}$ such that

$$\psi \quad = \quad \text{wp}[C](\varphi).$$

Idea: extend the syntax Exp to enable encoding proof rules for bounds on $\text{wp}[C](\varphi)$

$$\begin{array}{lclcl}
\text{wp}[\text{havoc } x](\varphi) & = & \forall x.\varphi & \rightsquigarrow & \mathcal{L}x.\ \varphi \\
\text{wp}[\texttt{assert } \psi](\varphi) & = & \psi \wedge \varphi & \rightsquigarrow & ? \\
\text{wp}[\text{assume } \psi](\varphi) & = & \psi \Rightarrow \varphi & \rightsquigarrow & ?
\end{array}$$

Our language should enable reasoning about *lower* and *upper* bounds

Expectations for Quantitative Conjunctions

Definition

$$\varphi \sqcap \psi \quad = \quad \lambda\sigma.\, \min\{\varphi(\sigma), \psi(\sigma)\}$$

New indicator function:

$$?(b) \quad = \quad [b] \cdot \infty \quad = \quad \lambda\sigma. \begin{cases} \infty, & \text{if } \sigma \models b \\ 0, & \text{otherwise} \end{cases}$$

Intuition: `true` and `false` are represented by ∞ and 0 in $\mathbb{R}^{\infty}_{\geq 0}$

Backward compatibility: $?(b_1 \wedge b_2) \quad = \quad ?(b_1) \ \sqcap \ ?(b_2)$

Expectations for Quantitative Implications

Definition

$$\varphi \Rightarrow \psi \quad = \quad \lambda\sigma. \begin{cases} \infty, & \text{if } \varphi(\sigma) \leq \psi(\sigma) \\ \psi(\sigma), & \text{otherwise} \end{cases}$$

Example

$$[\![?(b) \Rightarrow \varphi]\!](\sigma) \quad = \quad \begin{cases} [\![\varphi]\!](\sigma), & \text{if } \sigma \models b \\ \infty, & \text{otherwise} \end{cases}$$

Lemma (Adjointness of $\sqcap$ and $\Rightarrow$)

$$\rho \sqcap \varphi \preceq \psi \qquad \text{iff} \qquad \rho \preceq \varphi \Rightarrow \psi$$

The quantitative assertion language HeyLo

$$
\begin{array}{rcll}
\varphi & ::= & a & \text{(arithmetic expressions over rational variables)} \\
 & | & ?(b) & \text{(embedding of Boolean expressions)} \\
 & | & \varphi + \varphi & \text{(sums)} \\
 & | & \varphi \cdot \varphi & \text{(products)} \\
 & | & \varphi \sqcap \varphi & \text{(quantitative conjunction (minimum))} \\
 & | & \varphi \Rightarrow \varphi & \text{(quantitative implication)} \\
 & | & \mathcal{Z} x.\ \varphi & \text{(supremum quantifier over variable } x\text{)} \\
 & | & \mathcal{L} x.\ \varphi & \text{(infimum quantifier over variable } x\text{)} \\
 & | & \ldots & \text{(dual versions for upper bound reasoning)}
\end{array}
$$

Algebraic Facts

Definition

φ is **valid** iff $\forall \sigma.\ [\![\varphi]\!](\sigma) = \infty$

Theorem

$\varphi \preceq \psi$ iff $\varphi \Rightarrow \psi$ is valid

Definition

$$\neg\varphi \quad = \quad \varphi \Rightarrow 0 \quad = \quad \lambda\sigma. \begin{cases} \infty, & \text{if } [\![\varphi]\!](\sigma) = 0 \\ 0, & \text{otherwise} \end{cases}$$

Example

$$\nabla(\varphi) \quad = \quad \neg\neg\varphi \quad = \quad \lambda\sigma. \begin{cases} 0, & \text{if } [\![\varphi]\!](\sigma) = 0 \\ \infty, & \text{otherwise} \end{cases}$$

$(\mathsf{Exp}, \sqcap, \Rightarrow, \neg, 0, \infty)$ is a Heyting algebra (hence the name HeyLo)

Dual HeyLo Constructs

Main idea: construct dual Heyting algebra $(\mathsf{Exp}, \sqcup, \leftsquigarrow, \sim, \infty, 0)$ with analogous properties

$$0 \quad \rightsquigarrow \quad \texttt{true} \qquad \text{and} \qquad \infty \quad \rightsquigarrow \quad \texttt{false}$$

Co-conjunction

$$\varphi \sqcup \psi \quad = \quad \lambda\sigma.\, \max\{\varphi(\sigma), \psi(\sigma)\}$$

Coimplication

$$\varphi \leftsquigarrow \psi \quad = \quad \lambda\sigma. \begin{cases} 0, & \text{if } [\![\varphi]\!](\sigma) \geq [\![\psi]\!](\sigma) \\ [\![\psi]\!](\sigma), & \text{otherwise} \end{cases}$$

Co-negation

$$[\![\sim\varphi]\!] \quad = \quad \lambda\sigma. \begin{cases} 0, & \text{if } [\![\varphi]\!](\sigma) = \infty \\ \infty, & \text{otherwise .} \end{cases}$$

Double co-negation

$$[\![\triangle(\varphi)]\!] \quad = \quad [\![\sim\sim\varphi]\!] \quad = \quad \lambda\sigma. \begin{cases} \infty, & \text{if } [\![\varphi]\!](\sigma) = \infty \\ 0, & \text{otherwise} \end{cases}$$

What are those HeyLo formulae good for?

Rule: If $\Phi_\varphi(I) \preceq I$ then $\mathrm{wp}[\texttt{while}\ (b)\ \{\ C\ \}](\varphi) \ = \ \mathit{lfp}(\Phi_\varphi) \ \preceq \ I$

Verification condition: [Navarro & Olmedo, 2022]

$$\mathrm{vc}[\texttt{while}\ (b)\ \ \texttt{invariant}\ I\ \{\ C\ \}](\varphi) \quad = \quad \begin{cases} I, & \text{if } \Phi_\varphi(I) \preceq I \\ 0, & \text{otherwise} \end{cases}$$

Corresponding HeyLo formula:

$$\underbrace{\wr x_1.\ \ldots\ \wr x_n.\ \triangle(\Phi_\varphi(I) \Rightarrow I)}_{\text{evaluate to 0 if } \Phi_\varphi(I) \not\preceq I} \qquad \underbrace{\sqcap}_{\text{and}} \qquad \underbrace{I}_{\text{evaluate to invariant otherwise}}$$

Towards a verification infrastructure for probabilistic programs

1. **What are quantitative assertions?**

   $\rightsquigarrow$ **HeyLo formulae**, e.g. $(\mathcal{L}\, x_1.\ \ldots\ \mathcal{L}\, x_n.\ \triangle(\Phi_\varphi(I) \Rightarrow I)) \sqcap I$

2. **What is an intermediate language for probabilistic program verification?**

3. **What can be encoded in such a language?**

4. **Status of Mechanization and Automation**

The Intermediate Verification Language HeyVL

Ingredients of HeyVL: Loop-free pGCL

+ Boogie-like verification-specific commands
+ `validate` for enforcing conditions of proof rules
+ Dual versions, e.g. for upper-bound reasoning
+ Rewards for reasoning about resource consumption

Semantics: wp-style verification condition generator

$$\mathrm{vc}[C] \colon \mathrm{HeyLo} \to \mathrm{HeyLo}$$

The Intermediate Verification Language HeyVL [OOPSLA 2023]

C	$\mathrm{vc}[C](\varphi)$	dual $\mathrm{vc}[co\ldots](\varphi)$
$x := \mu$	$\mathrm{wp}[x := \mu](\varphi)$	
$C_1 ; C_2$	$\mathrm{vc}[C_1](\,\mathrm{vc}[C_2](\varphi)\,)$	
C_1 `[]` C_2	$\mathrm{vc}[C_1](\varphi) \sqcap \mathrm{vc}[C_2](\varphi)$	$\mathrm{vc}[C_1](\varphi) \sqcup \mathrm{vc}[C_2](\varphi)$
`assert` ψ	$\psi \sqcap \varphi$	$\psi \sqcup \varphi$
`assume` ψ	$\psi \Rightarrow \varphi$	$\psi \leftsquigarrow \varphi$
`havoc` x	$\wp x.\ \varphi$	$\mathcal{Z} x.\ \varphi$
`validate`	$\triangle(\varphi)$	$\triangledown(\varphi)$
`reward` a	$a + \varphi$	

Example: lower bound reasoning for weakest preexpectations

Given: pGCL program $C \in$ HeyVL expectations $\varphi, \psi \in$ HeyLo

$$
\begin{aligned}
& \psi \preceq \mathsf{wp}[C](\varphi) \\
\text{iff} \quad & \psi \preceq \mathsf{vc}[C](\varphi) \\
\text{iff} \quad & \psi \preceq \mathsf{vc}[C](\varphi \sqcap \infty) \\
\text{iff} \quad & \psi \Rightarrow \mathsf{vc}[C](\varphi \sqcap \infty) \ \text{valid} \\
\text{iff} \quad & \mathsf{vc}[\texttt{assume}\ \psi;\ C;\ \texttt{assert}\ \varphi](\infty) \ \text{valid}
\end{aligned}
$$

$\rightsquigarrow$ Lower bound reasoning reduces to checking validity

$\rightsquigarrow$ Upper bound reasoning dually reduces to checking covalidity

Towards a verification infrastructure for probabilistic programs

1. **What are quantitative assertions?**

   $\rightsquigarrow$ **HeyLo formulae**, e.g. $(\text{ʃ}\, x_1.\ \ldots\ \text{ʃ}\, x_n.\ \triangle(\Phi_f(I) \Rightarrow I)) \sqcap I$

2. **What is an intermediate language for probabilistic program verification?**

   $\rightsquigarrow$ **HeyVL** $\approx$ pGCL + dual Boogie-like verification-specific commands

3. **What can be encoded in such a language?**

4. **Status of Mechanization and Automation**

Example: Loop Invariants for Partial Correctness

Given: **while** (b) { C } with modified variables $x_1, \ldots, x_n$

Characteristic function: $\Phi_\psi(I) \;=\; [b] \cdot \mathrm{wlp}[C](I) \;+\; [\neg b] \cdot \psi$

Proof rule: If $I \;\preceq\; \Phi_\psi(I)$ then $\mathrm{wlp}[\textbf{while}\ (b)\ \{\ C\ \}](\psi) \;=\; gfp(\Phi_\psi) \;\succeq\; I$

```
assert I;
havoc x_1, ..., x_n;
validate;
assume I;
if (b) {
  C;
  assert I;
  assume ?(false)
} else { }   /// ψ
```

Soundness of HeyVL encoding

$$
\begin{aligned}
& \mathrm{wlp}[\textbf{while}\ (b)\ \{\ C\ \}](\psi) \\
\succeq\ & \mathrm{vc}[\mathit{encoding}](\psi) \\
=\ & \ell x_1.\ \ldots\ \ell x_n.\ \triangle(I \Rightarrow \Phi_\psi(I)) \sqcap I \\
=\ & \begin{cases} I, & \text{if } \Phi_\psi(I) \succeq I \\ 0, & \text{otherwise} \end{cases}
\end{aligned}
$$

Proof rules that have been automated with HeyVL

Problem	Verification Technique	Source
LPROB	wlp + Park induction	[McIver & Morgan, 2005]
	wlp + latticed k-induction	[OOPSLA 2023]
UEXP	wp + Park induction	[McIver & Morgan, 2005]
	wp + latticed k-induction	[CAV 2021]
LEXP	wp + ω-invariants	[Kaminski, 2019]
	wp + Optional Stopping Theorem	[Hark et al., 2019]
CEXP	wp + conditioning	[Olmedo et al., 2018]
LERT	ert calculus + ω-invariants	[ESOP, 2016]
AST	parametric super-martingales	[McIver et al., 2018]
PAST	program analysis with martingales	[Chakarov & Sankaranarayanan, 2013]
UEXP	new rule for continuous distributions	[Batz et al., 2025]
AST	new rule for distributed systems	[Enea et al., 2026]

Towards a verification infrastructure for probabilistic programs

1. **What are quantitative assertions?**

   $\rightsquigarrow$ **HeyLo formulae**, e.g. $(\text{ʃ}\, x_1.\ \ldots\ \text{ʃ}\, x_n.\ \triangle(\Phi_f(I) \Rightarrow I)) \sqcap I$

2. **What is an intermediate language for probabilistic program verification?**

   $\rightsquigarrow$ **HeyVL** $\approx$ pGCL + dual Boogie-like verification-specific commands

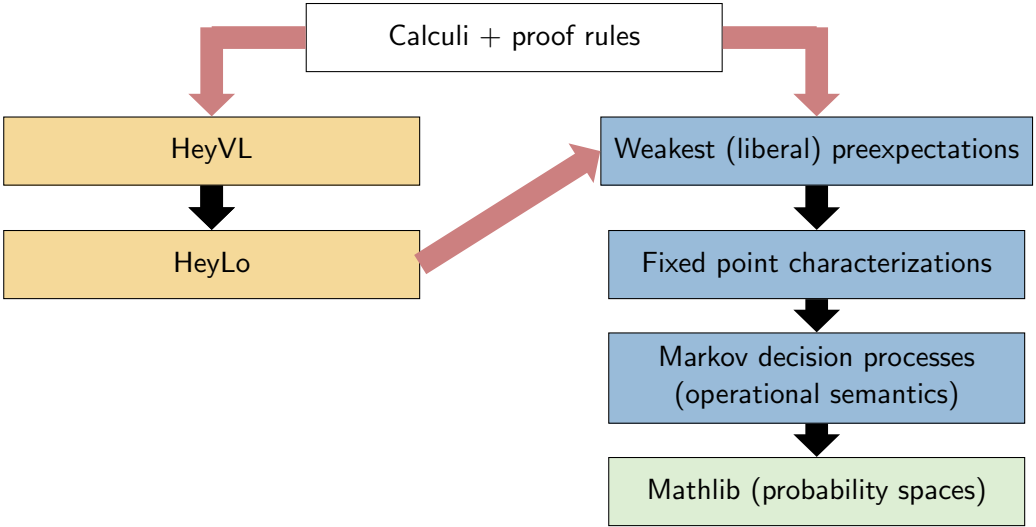
3. **What can be encoded in such a language?**

   $\rightsquigarrow$ **many proof rules based on expectations or supermartingales**

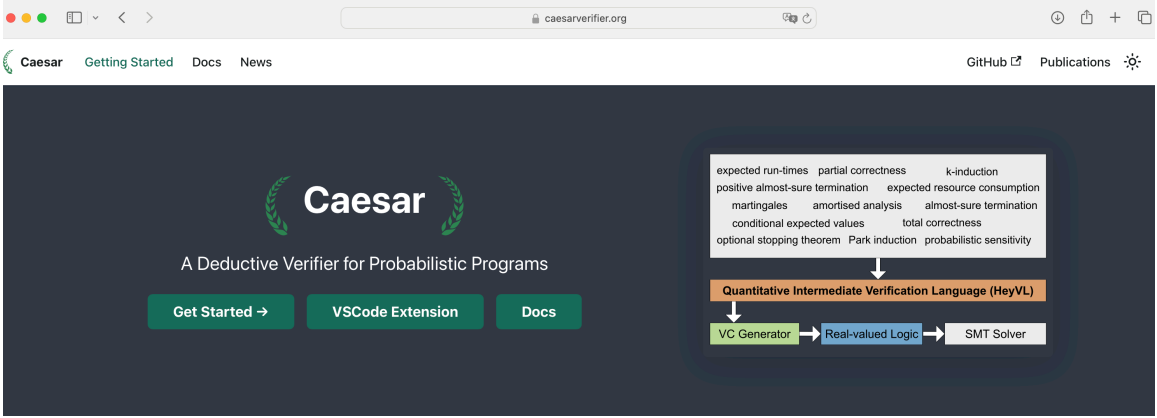
4. **Status of Mechanization and Automation**

Lean Formalization

(10k LOC) { ITP 2026 } [ε] { *arXiv* }



Caesar: An Automated Verifier for HeyVL



caesarverifier.org

Caesar: An Automated Verifier for HeyVL

$\sim$13k LOC of Rust code

- Backend 1: Verification condition generator + SMT (z3)
- Backend 2: Bounded probabilistic model checking via MDP semantics
- Frontend for simple weakest preexpectation calculi
- Various language extensions (custom types and distributions...)

Performance is competitive with specialized tools demonstrating new proof rules

- Custom rewritings for dealing with ∞
- Quantifier elimination for $\mathcal{L}$, $\mathcal{Z}$

Caesar enables rapid prototyping of new proof rules for probabilistic programs

What are concrete programs verified with Caesar?

Bounded Retransmission Protocol [Helmink et al.'93, D'Argenio et al.'97]

- Try to send N packets via a lossy channel
- Transmitting a single packet fails with probability p
- Attempt at most F retransmissions per packet; otherwise abort

$$
\begin{aligned}
& \mathit{sent} := 0;\ \mathit{fail} := 0; \\
& \texttt{while}\ (\mathit{sent} < N \ \wedge\ \mathit{fail} < F)\ \{ \\
& \quad \{\underbrace{\mathit{fail} := \mathit{fail} + 1}_{\text{failed transmission}}\}\ [p]\ \{\underbrace{\mathit{fail} := 0;\ \mathit{sent} := \mathit{sent} + 1}_{\text{successful transmission}}\} \\
& \}
\end{aligned}
$$

- Verified properties: upper bounds on the expected number of transmissions
- Encoded technique: Latticed k-Induction [CAV 2021]

Variant of Random Walk

$$
\begin{array}{l}
\texttt{while}\ (x > 0)\ \{ \\
\quad q\ :=\ x/(2 \cdot x + 1); \\
\quad \{x\ :=\ x - 1\}\ [q]\ \{x\ :=\ x + 1\} \\
\}
\end{array}
$$

- Verified property: almost-sure termination
- Encoded technique: parametric supermartingales [McIver et al., 2017]

Coupon Collector's Problem

```
while (0 < x) {
  i := N + 1;
  while (0 < x < i) {
    i :≈ unif(1, N)
  }
  x := x − 1
}
```

- Verified property: expected runtime $\leq N \cdot \mathcal{H}(N) \quad = \quad N \cdot \sum_{k=1}^{N} 1/k$
- Encoded technique: expected runtime calculus [JACM 2018]

Conclusion

An infrastructure for automating verification of probabilistic programs

1. **What are quantitative assertions?**
   - $\rightsquigarrow$ **HeyLo formulae**, e.g. $(\wr x_1. \ \ldots \ \wr x_n. \triangle(\Phi_f(I) \Rightarrow I)) \sqcap I$
2. **What is an intermediate language for probabilistic program verification?**
   - $\rightsquigarrow$ **HeyVL** $\approx$ pGCL + dual Boogie-like verification-specific commands
3. **What can be encoded in such a language?**
   - $\rightsquigarrow$ **many proof rules based on expectations or supermartingales**
4. **Status of Mechanization and Automation**
   - $\rightsquigarrow$ **Caesar: automated verification tool using SMT or probabilistic model checking**
   - $\rightsquigarrow$ **Lean mechanization of HeyVL + MDPs + framework for expectation calculi**
     - $\rightsquigarrow$ **Soundness proofs for some existing calculi and proof rules**
     - $\rightsquigarrow$ **WIP: tactics for interactive verification**

Ongoing and Future Work

- Better tactics for interactive reasoning about PPs in Lean
- Leveraging stochastic indepence à la PSL, Lilac, Bluebell $\rightsquigarrow$ Roberto Pettinau
- Extension to weighted programming $\rightsquigarrow$ Oliver Bøving
  - Weights given by semirings instead of probabilities
  - Application: verification of software defined networks via ProbNetKAT (see PLDI 2026)